SDG BLOCKCHAIN ACCELERATOR

Prototype (PoC) Report

1. Project Information

- **Project Name:** Digital Infrastructure for Green Energy Exports / Fuel Switch x UNDP BiH Pilot Cohort 2
- **Challenge & UNDP Office:** Bosnia & Herzegovina Country Office
- **Document Version: 1.0.0**

2. Project Overview

This prototype facilitates the automated issuance and retirement of Renewable Energy Certificates (RECs) on the Cardano blockchain.

- Minting: Converts verified solar energy generation data into NFTs. Each NFT represents a maximum of 1000kWh to ensure granular tracking.
- Validation: An Aiken smart contract ensures that only an authorized admin can mint, that the energy values are within allowed limits, and that the asset names follow a strict, transparent metadata format.
- Retirement: Provides a secure "Burn" mechanism to retire certificates once the renewable energy claim is used, preventing double-counting.

3. Repository Structure

Structure:

/nft-marketplace: Root directory for the Aiken project.

- /validators: Contains the smart contract source code, specifically rec_nft_mint.ak.
- aiken.toml: Project configuration file including compiler version (v1.1.19) and Plutus version (V3).
- plutus.json: The compiled blueprint containing the CBOR hex of the validator for backend use.

/backend : Contains the Node.js Express server (server.js), Redis configuration, and Lucid integration scripts.

/docs: Contains the PRD, API Reference, and User Manual

4. Build Instructions

- To compile the smart contracts and generate the necessary artifacts for the backend:
  - Install Aiken CLI: Ensure Aiken v1.1.19 is installed.
  - Compile Contract: Navigate to the project root and run:
    aiken build
    This generates the plutus.json file in the root directory.
  - Derive Policy ID: Use the blueprint to calculate the unique Policy ID:
    aiken blueprint policy
    This Policy ID is then used in the backend environment variables (.env).

5. Test Instructions

- The prototype includes a native Aiken testing suite to verify all functional requirements:
- Run Unit Tests: Execute the following command to run the 8+ defined test scenarios: aiken check
- Validated Edge Cases:
  - mint_02_fail_kw_too_high: Confirms the contract rejects any token representing >1000 kWh.
  - mint_03_fail_name_mismatch: Ensures the asset name strictly matches the redeemer metadata.
  - burn_03_fail_positive_mint: Verifies that the "Burn" action cannot be exploited to mint new tokens

6. Deployment Instructions

Prerequisites:

- Aiken CLI: v1.1.19 installed.
- Node.js: v18+ installed.
- Redis: A running instance (local or cloud) for idempotency checks.
- Cardano Access: A Blockfrost Project ID (Preprod).

- Wallet: A "Hot Wallet" seed phrase funded with 100 tADA.

Step 1: Compile Smart Contracts

- Before the backend can construct transactions, the Aiken validator must be compiled to generate the plutus.json blueprint.

```
# Navigate to the Aiken directory

cd nft-marketplace

# Build the contract and generate artifacts

aiken build

# Output: ./plutus.json (Contains the CBOR hex of the validator)
```

Step 2: Derive Policy ID

- The Policy ID is required for the backend environment variables to identify assets minted by this contract.

```
# Calculate the Policy ID from the compiled blueprint

aiken blueprint policy

# Copy the output string (e.g 524543) for use in the .env file
```

Step 3: Backend Environment Configuration

Configure the Node.js server to interact with the Cardano Preprod network. Create a .env file in the root backend directory:

```
BLOCKFROST_PROJECT_ID=preprod...       # Your Blockfrost Key

WALLET_SEED="your 24 word phrase..."   # Hot Wallet Mnemonic

INTERNAL_API_KEY=FSDCYK...             # Security Key for API Access
```

```
REDIS_URL=redis://localhost:6379       # Connection to Redis

POLICY_ID=<Paste Policy ID from Step 2>
```

Step 4: Deploy & Run API Service

The backend acts as the deployment vehicle, serving the smart contract logic via API endpoints.

```
# Install dependencies (including @lucid-evolution/lucid)

npm install

# Start the server (Local or Production)

npm start

# Server should listen on Port 3000 (or PORT env var)
```

Step 5: Verify Deployment (First Mint)

To "deploy" the first asset (Minting Policy execution), trigger the /mint endpoint using curl. This builds, signs, and submits the transaction via Lucid.

```
curl -X POST http://localhost:3000/mint \

  -H "Content-Type: application/json" \

  -H "x-api-key: FSDCYKJHGYU78SDFGHJK23456789TYUI" \

  -d '{

    "metadata": {

      "name": "DeploymentTest",

      "energy_kwh": 1000,

      "vintage_start": 1763712343,

      "vintage_end": 1763712350
```

```
    }
  }'
Expected Output:
{
  "status": "success",
  "txHashes": ["<Transaction_Hash>"],
  "policyId": "<Your_Policy_ID>"
}
```

7. Testnet / Emulator Results

Transaction ID	Contract Action	Status	Notes
tx_mint_01	Mint Batch (15 NFTs)	SUCCESS	Batch ID used for uniqueness
tx_burn_01	Burn REC	SUCCESS	Successfully retired 1 certificate
tx_fail_02	Mint(No Admin)	FAILED	Rejected: Admin signature missing

8. Dependencies & Environment

- Aiken CLI: v1.1.0
- Cardano Network: Preprod Testnet
- Off-chain SDK: Lucid Evolution

- Infrastructure: Blockfrost API, Redis (Idempotency layer)
- Backend: Express.js (Node.js)

9. Demo / Walkthrough

- Videos Included ~Cardano integration and Platform demo

10. Remaining Issues / Next Steps

- Optimization: Implement a more efficient way to handle the list.all check in Aiken to further reduce execution costs for very large batches (e.g., 50+ NFTs).
- Architectural Pivot: Transition from individual NFTs to Fungible Tokens (FTs) to allow bulk minting in a single transaction, significantly reducing fees.
- Persistence Tuning: Configure Redis with AOF (Append Only File) persistence to ensure the idempotency cache is not lost during server restarts.
- Performance: Optimize the "Transaction Chunking" loop to maximize the limit of 15 assets per transaction under varied network conditions.